

Program Designing 'and' { [Programming].ICT 1305

< J.M.D.S.Jayasinghe >

< Department of ICT >

< Faculty of Technology >

< Rajarata University of Sri Lanka >

}

What is an Algorithm {

- Algorithms are the threads that tie together most of the subfields of computer science.
- An algorithm is a step-by-step procedure or set of rules designed to perform a specific task or solve a particular problem.
- It is a logical and systematic way of accomplishing an objective, often used in mathematics, computer science, and other fields.

}

How do Algorithms Work? {

- **Input:** Algorithms take input data, which can be in various formats, such as numbers, text, or images.
- **Processing:** The algorithm processes the input data through a series of logical and mathematical operations, manipulating and transforming it as needed.
- **Output:** After the processing is complete, the algorithm produces an output, which could be a result, a decision, or some other meaningful information.

}

How do Algorithms Work? {

- **Efficiency:** A key aspect of algorithms is their efficiency, aiming to accomplish tasks quickly and with minimal resources.
- **Optimization:** Algorithm designers constantly seek ways to optimize their algorithms, making them faster and more reliable.
- **Implementation:** Algorithms are implemented in various programming languages, enabling computers to execute them and produce desired outcomes.

}

Properties of an Algorithm {

- **Finiteness:** The algorithm must always terminate after a finite number of steps.
- **Definiteness:** Each step must be precisely defined; the actions to be carried out must be rigorously and unambiguously specified for each case.
- **Input:** An algorithm has zero or more inputs, taken from a specified set of objects.

}

Properties of an Algorithm {

- **Output:** An algorithm has one or more outputs, which have a specified relation to the inputs.
- **Effectiveness:** All operations to be performed must be sufficiently basic that they can be done exactly and in finite length.

}

Types of Algorithms {



Search engine
algorithm



Encryption
algorithm



Greedy
algorithm



Recursive
algorithm



Backtracking
algorithm



Divide-
and-conquer
algorithm



Dynamic
programming
algorithm



Brute-force
algorithm



Sorting
algorithm



Hashing
algorithm



Randomized
algorithm

}

Types of Algorithms {

- **Brute Force Algorithm:** A straightforward approach that exhaustively tries all possible solutions, suitable for small problem instances but may become impractical for larger ones due to its high time complexity.
- **Searching Algorithm:** Designed to find a specific target within a dataset, enabling efficient retrieval of information from sorted or unsorted collections.

}

Types of Algorithms {

- **Recursive Algorithm:** A method that breaks a problem into smaller, similar subproblems and repeatedly applies itself to solve them until reaching a base case, making it effective for tasks with recursive structures.
- **Greedy Algorithm:** Makes locally optimal choices at each step in the hope of finding a global optimum, useful for optimization problems but may not always lead to the best solution.

}

Types of Algorithms {

- **Encryption Algorithm:** Utilized to transform data into a secure, unreadable form using cryptographic techniques, ensuring confidentiality and privacy in digital communications and transactions.
- **Dynamic Programming Algorithm:** Stores and reuses intermediate results to avoid redundant computations, enhancing the efficiency of solving complex problems.

}

Types of Algorithms {

- **Divide and Conquer Algorithm:** Breaks a complex problem into smaller subproblems, solves them independently, and then combines their solutions to address the original problem effectively.
- **Sorting Algorithm:** Aimed at arranging elements in a specific order, like numerical or alphabetical, to enhance data organization and retrieval.

}

Types of Algorithms {

- **Backtracking Algorithm:** A trial-and-error technique used to explore potential solutions by undoing choices when they lead to an incorrect outcome, commonly employed in puzzles and optimization problems.
- **Hashing Algorithm:** Converts data into a fixed-size hash value, enabling rapid data access and retrieval in hash tables, commonly used in databases and password storage.

}

Use of the Algorithms {

Data Analysis and Machine Learning

- Algorithms in data analysis and machine learning identify patterns in large datasets and predict outcomes.
- Techniques like support vector machines, decision trees, and neural networks enable computers to learn and improve, supporting applications such as recommendation systems, natural language processing, and image recognition.

}

Use of the Algorithms {

Cryptography & Security

- Cryptography algorithms, like SHA-256, AES, and RSA, protect data through encryption and decryption, ensuring secure storage, communication, and online transactions.
- They underpin cyber security measures for data integrity, user authentication, and sensitive information security.

}

Use of the Algorithms {

Information Retrieval and Search Engines

- Search algorithms like PageRank and Hummingbird enable search engines to index and retrieve relevant information efficiently.
- They rank web pages by importance and relevance, helping users find the most pertinent content amid vast online data.

}

Use of the Algorithms {

Optimization problems

- Optimization methods, such as gradient descent, linear programming, and genetic algorithms, identify the best solutions to complex problems.
- Widely used in industries like finance, engineering, logistics, and AI, they enhance productivity, reduce costs, and optimize resource use for operations and decision-making.

}

Use of the Algorithms {

Genomics and medical diagnostics

- Algorithms play a vital role in medical diagnostics by analyzing images, predicting disease outbreaks, and identifying genetic changes.
- Machine learning has revolutionized personalized medicine, enabling tailored treatments based on genetic profiles, while also accelerating genomic data sequencing and interpretation, advancing biotechnology and genomics research.

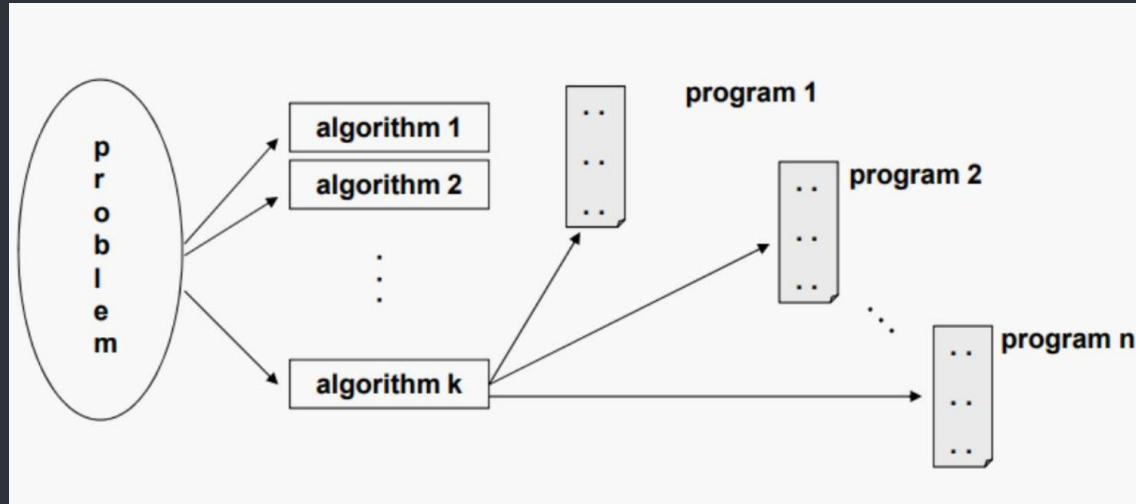
}

Problems vs Algorithms vs Programs {

- For each problem or class of problems, there may be many different algorithms.
- For each algorithm, there may be many different implementations (programs).
- A concrete implementation of an algorithm in a programming language that can be executed by a computer.
- The machine can't read a program directly, because it only understands machine code.
- Write stuff in a computer language, and then a compiler or interpreter can make it understandable to the computer.

}

Problem Solving {



Expressing Algorithms {

- **Natural Language:** usually verbose and ambiguous
- **Flowcharts:** avoid most (if not all) issues of ambiguity; difficult to modify without specialized tools; largely standardized
- **Pseudo-code:** also avoids most issues of ambiguity; vaguely resembles common elements of programming languages; no particular agreement on syntax
- **Programming Language:** tend to require expressing low level details that are not necessary for a high-level understanding

}

Common Elements of Algorithms {

Acquire data (input)

Reading values from an external source; most algorithms require data values to define the specific problem.

Computation

Some methods of performing arithmetic calculations, comparisons, checking logical conditions, etc.

Selection

A method of choosing between two or more courses of action, based on initial data, user input, and/or computerized results.

}

Common Elements of Algorithms {

Iteration

A method of repeatedly executing a set of instructions a specified number of times or until some logical condition is reached.

Report results (output)

Some way to report computerized results to the user or request additional data from the user.

}

How to Write an Algorithm? {

- There are no well-defined standards for writing algorithms.
- An algorithm can be written using these common basic code constructs (do, for, while, if, else).
- Algorithms are typically written in a step-by-step fashion.
- Algorithm writing is a process that occurs after the problem domain has been well-defined.

1. Understand the Problem

2. Plan the Logic

3. Write the Steps

4. Verify and Test the algorithm

}

Algorithm to make a cup of tea {

1. Fill up the kettle with water
2. Boil the water
3. Place a teabag in your favourite mug
4. Pour boiling water into your favourite mug
5. Brew the tea for a few moments
6. Remove and dispose of the teabag
7. Add milk
8. Add sugar
9. Stir the tea
10. Enjoy the hot beverage



Algorithm to make a cake {

- **Problem:** Make a chocolate cake.
- **Algorithm:** Step-by-step instructions in the recipe.
 - Preheat the oven to 180°C.
 - Mix flour, sugar, cocoa, and eggs.
 - Pour the batter into a pan.
 - Bake for 30 minutes.
 - Check with a toothpick; if done, cool and serve.
- **Input:** Ingredients (flour, sugar, eggs, cocoa).
- **Output:** Chocolate cake.
- **Why it's an algorithm:** Follows precise steps and produces a definite result.

}

Algorithm to making a sandwich {

1. Take two slices of bread.
2. Spread butter or mayonnaise on one slice.
3. Add lettuce, tomato, or other vegetables.
4. Add slices of cheese or meat.
5. Place the second slice of bread on top.
6. Cut the sandwich into halves if desired.
7. Serve on a plate and enjoy.

}

1

2

3

4

5

6

7

8

9

10

11

12

13

14

Let's Apply
Algorithmic concepts
into Computing

Build an Algorithm {

Algorithm for Print Hello World 10 times

- 1.This problem is also solved using the loop concept.
- 2.Take a variable count and set it to zero.
- 3.Then print "Hello World" and increment count by 1.
- 4.Next, check if count is less than 10. If this is TRUE then again print "Hello World" and increment the variable count.
- 5.On the other hand if the condition if FALSE then execution will stop.

}

Build an Algorithm {

Algorithm (in simple English)

1. Initialize count = 0 (PROCESS)
2. Print Hello World (I/O)
3. Increment count by 1 (PROCESS)
4. Is count < 10 (DECISION)
5. If YES go to step 2
6. else Stop

}

Build an Algorithm {

Step-by-Step Algorithm

Step 1: Start.

Step 2: Set a counter variable to 1.

Step 3: Repeat the following steps (4, 5) while the counter is less than or equal to 10:

Step 4: Print "Hello World".

Step 5: Increment the counter by 1.

Step 6: End.

}

Build an Algorithm {

Algorithm for Add 10 and 20

1. To solve this problem we will take a variable sum and set it to zero.
2. Then we will take the two numbers 10 and 20 as input.
3. Next we will add both the numbers and save the result in the variable sum i.e., $\text{sum} = 10 + 20$.
4. Finally, we will print the value stored in the variable sum.

}

Build an Algorithm {

Algorithm (in simple English)

1. Initialize sum = 0 (PROCESS)
2. Enter the numbers (I/O)
3. Add them and store the result in sum (PROCESS)
4. Print sum (I/O)

}

Build an Algorithm {

Step-by-Step Algorithm

Step 1: Start.

Step 2: Take the first number as 10.

Step 3: Take the second number as 20.

Step 4: Add the first number and the second number.

Step 5: Store the result.

Step 6: Display the result.

Step 7: End.

}

Build an Algorithm {

Algorithm to find the average of 3 subjects

Step 1: Start the Program

Step 2: Declare and Read 3 Subject, (S1, S2, S3)

Step 3: Calculate the sum of all the 3 Subject values and store result in Sum variable (Sum = S1+S2+S3)

Step 4: Divide Sum by 3 and assign it to Average variable.
(Average = Sum/3)

Step 5: Print the value of Average of 3 Subjects

Step 6: End the Program

}

Build an Algorithm {

Algorithm for multiplies two numbers and displays the output.

Step 1: Start

Step 2: declare three integers x, y & z

Step 3: define values of x & y

Step 4: multiply values of x & y

Step 5: store result of step 4 to z

Step 6: print z

Step 7: Stop

}

{ Thank You }